How Command Palette extensions work

Command Palette extensions are standalone .NET applications that communicate with Command Palette through a WinRT API. Each extension runs in its own process and registers with Command Palette through its app manifest, making extensions isolated, secure, and easy to deploy.

This page explains the core concepts behind the extension model — how extensions are discovered, how they provide commands and pages, and what kinds of experiences you can build.

How Command Palette discovers installed extensions

Command Palette uses the Windows Package Catalog to find installed apps that declare themselves as Command Palette extensions. Extensions register through their `.appxmanifest` by declaring a `windows.appExtension` with the name `com.microsoft.commandpalette`:

XML

```
<Extensions>
    <com:Extension Category="windows.comServer">
        <com:ComServer>
            <com:ExeServer Executable="ExtensionName.exe" Arguments="-
RegisterProcessAsComServer" DisplayName="Sample Extension">
                <com:Class Id="<Extension CLSID Here>" DisplayName="Sample
Extension" />
            </com:ExeServer>
        </com:ComServer>
    </com:Extension>
    <uap3:Extension Category="windows.appExtension">
        <uap3:AppExtension Name="com.microsoft.commandpalette"
                           Id="YourApplicationUniqueId"
                           PublicFolder="Public"
                           DisplayName="Sample Extension"
                           Description="Sample Extension for Command Palette">
            <uap3:Properties>
                <CmdPalProvider>
                    <Activation>
                        <CreateInstance ClassId="<Extension CLSID Here>" />
                    </Activation>
                    <SupportedInterfaces>
                        <Commands />
                    </SupportedInterfaces>
```

```
            </CmdPalProvider>
          </uap3:Properties>
        </uap3:AppExtension>
    </uap3:Extension>
</Extensions>
```

Extensions use an out-of-process COM server as the communication layer between your app and Command Palette. **Don't worry about the details** — the template project handles creating the COM server, starting it, and marshalling your objects to Command Palette automatically.

Important notes about the manifest

- The `AppExtension` must set `Name` to `com.microsoft.commandpalette`. This is the unique identifier that Command Palette uses to discover extensions.
- The `ComServer` element registers a COM class GUID that Command Palette uses to instantiate your extension. Make sure this CLSID is unique and matches across all three locations in the manifest.
- The `CmdPalProvider` element in the `Properties` section specifies the CLSID of the COM class that Command Palette will instantiate. Currently, only `Commands` is supported.

The extension API

Command Palette defines a WinRT API (Microsoft.CommandPalette.Extensions) that extensions use to communicate with Command Palette. A companion toolkit library (Microsoft.CommandPalette.Extensions.Toolkit) provides base classes and helpers that simplify common patterns.

Every extension implements the `IExtension` interface, which provides a `GetProvider` method that returns your `ICommandProvider`. The command provider is where you define the commands and pages that your extension offers.

Commands and pages

Extensions can provide several types of content:

- **Top-level commands** — Commands that appear on the Command Palette home page, making them immediately accessible to users.

- **Fallback commands** — Commands that are triggered when no other results match a user's query, useful for search-based or catch-all functionality.
- **Context menu items** — Additional actions that appear in the right-click context menu of other commands.

Each command can navigate to a **page** that displays content. Command Palette supports the following page types:

Expand table

Page type	Description
List pages	Display a searchable list of selectable items.
Detail pages	Show rich content with sections, tags, and links.
Form pages	Present user input fields for interactive workflows.
Markdown pages	Render formatted markdown content.
Grid pages	Display items in a gallery or grid layout.

Extensions can also provide their own settings pages for per-extension configuration, and support pinning commands to the Dock.

Get started

Ready to build your first extension? Head to Getting started to set up your project and create your first command.

Related content

- Getting started
- Extension samples
- API reference
- PowerToys Command Palette utility

Last updated on 04/16/2026